

ECE 4502/6502 & CS 6501 Machine Learning on Graphs: Homework 3

Due: 04/21/2025 23:59 PM EST

1. Node Embeddings with TransE (25 points)

While many real world systems are effectively modeled as graphs, graphs can be a cumbersome format for certain downstream applications, such as machine learning models. It is often useful to represent each node of a graph as a vector in a continuous low dimensional space. The goal is to preserve information about the structure of the graph in the vectors assigned to each node.

Multi-relational graphs are graphs with multiple types of edges. They are incredibly useful for representing structured information, as in knowledge graphs. There may be one node representing “Washington, DC” and another representing “United States,” and an edge between them with the type “Is capital of.” In order to create an embedding for this type of graph, we need to capture information about not just which edges exist, but what the types of those edges are. In this problem, we will explore a particular algorithm designed to learn node embeddings for multi-relational graphs.

The algorithm we will look at is TransE. We will first introduce some notation used in the paper describing this algorithm. We’ll let a multi-relational graph $G = (E, S, L)$ consist of: E : the set of entities (nodes), S : the set of edges, and L : the set of possible relationships.

The set S consists of triples (h, ℓ, t) , where $h \in E$ is the head or source-node, $\ell \in L$ is the relationship, and $t \in E$ is the tail or destination-node. As a node embedding, TransE tries to learn embeddings of each entity $e \in E$ into $\mathbb{R}^k$ (k-dimensional vectors), which we will denote by $\mathbf{e}$. The main innovation of TransE is that each relationship ℓ is also embedded as a vector $\ell \in \mathbb{R}^k$, such that the difference between the embeddings of entities linked via the

relationship ℓ is approximately ℓ . That is, if $(h, \ell, t) \in S$, TransE tries to ensure that:

$$\mathbf{h} + \ell \approx \mathbf{t}.$$

Simultaneously, TransE tries to make sure that:

$$\mathbf{h} + \ell \not\approx \mathbf{t}$$

if the edge (h, ℓ, t) does not exist.

Note on notation: We will use unbolded letters h, ℓ , etc., to denote the entities and relationships in the graph, and bold letters $\mathbf{h}, \ell$, etc., to denote their corresponding embeddings. TransE accomplishes this by minimizing the following loss:

$$L = \sum_{(h, \ell, t) \in S} \sum_{(h', \ell, t') \in S'_{(h, \ell, t)}} [\gamma + d(\mathbf{h} + \ell, \mathbf{t}) - d(\mathbf{h}' + \ell, \mathbf{t}')]_+. \quad (1)$$

Here, (h', ℓ, t') are "corrupted" triplets, chosen from the set $S'_{(h, \ell, t)}$ of corruptions of (h, ℓ, t) , which are all triples where either h or t (but not both) is replaced by a random entity:

$$S'_{(h, \ell, t)} = \{(h', \ell, t) \mid h' \in E\} \cup \{(h, \ell, t') \mid t' \in E\}.$$

Additionally, $\gamma > 0$ is a fixed scalar called the margin, the function $d(\cdot, \cdot)$ is the Euclidean distance, and $[x]_+ = \max(0, x)$ denotes the positive part function. Finally, TransE restricts all the entity embeddings to have unit length:

$$\|\mathbf{e}\|_2 = 1 \quad \text{for every } e \in E.$$

Algorithm 1 Learning TransE

input Training set $S = \{(h, \ell, t)\}$, entities and rel. sets E and L , margin γ , embeddings dim. k .

```
1: initialize  $\ell \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$  for each  $\ell \in L$ 
2:    $\ell \leftarrow \ell / \|\ell\|$  for each  $\ell \in L$ 
3:    $\mathbf{e} \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$  for each entity  $e \in E$ 
4: loop
5:    $\mathbf{e} \leftarrow \mathbf{e} / \|\mathbf{e}\|$  for each entity  $e \in E$ 
6:    $S_{batch} \leftarrow \text{sample}(S, b)$  // sample a minibatch of size  $b$ 
7:    $T_{batch} \leftarrow \emptyset$  // initialize the set of pairs of triplets
8:   for  $(h, \ell, t) \in S_{batch}$  do
9:      $(h', \ell, t') \leftarrow \text{sample}(S'_{(h, \ell, t)})$  // sample a corrupted triplet
10:     $T_{batch} \leftarrow T_{batch} \cup \{((h, \ell, t), (h', \ell, t'))\}$ 
11:   end for
12:   Update embeddings w.r.t.  $\sum_{((h, \ell, t), (h', \ell, t')) \in T_{batch}} \nabla[\gamma + d(\mathbf{h} + \ell, \mathbf{t}) - d(\mathbf{h}' + \ell, \mathbf{t}')]_+$ 
```

```
13: end loop
```

1.1 Simplified Objective (3 points)

Say we were intent on using a simpler loss function. Our objective function (1) includes a term maximizing the distance between $\mathbf{h}' + \ell$ and $\mathbf{t}'$. If we instead simplified the objective, and just tried to minimize:

$$L_{\text{simple}} = \sum_{(h, \ell, t) \in S} d(\mathbf{h} + \ell, \mathbf{t}), \quad (2)$$

we would obtain a useless embedding.

Problem: Give an example of a simple graph and corresponding embeddings which will minimize the new objective function (2) all the way to zero, but still give a completely useless embedding.

Hint: Your graph should be non-trivial, i.e., it should include at least two nodes and at least one edge. Assume the embeddings are in 2 dimensions, i.e., $k = 2$. What happens if $\ell = 0$?

1.2 Utility of Margin γ (5 points)

We are interested in understanding what the margin term γ accomplishes. If we removed the margin term γ from our loss, and instead optimized:

$$L_{\text{no margin}} = \sum_{(h, \ell, t) \in S} \sum_{(h', \ell, t') \in S'_{(h, \ell, t)}} [d(\mathbf{h} + \ell, \mathbf{t}) - d(\mathbf{h}' + \ell, \mathbf{t}')]_+, \quad (3)$$

it turns out that we would again obtain a useless embedding. Give an example of a simple graph and corresponding embeddings which will minimize the new objective function (3) all the way to zero, but still give a completely useless embedding. By useless, we mean that in your example, you cannot tell just from the embeddings whether two nodes are linked by a particular relation. Your graph should be non-trivial, i.e., it should include at least two nodes and at least one edge. Assume the embeddings are in 2 dimensions, i.e., $k = 2$.

1.3 Normalizing the Embeddings (7 points)

Recall that TransE normalizes every entity embedding to have unit length (see line 5 of the algorithm). The quality of our embeddings would be much worse if we did not have this step. To understand why, imagine running the algorithm with line 5 omitted. What could the algorithm do to trivially minimize the loss in this case? What would the embeddings it generates look like?

1.4 Expressiveness of TransE Embeddings (10 points)

Give an example of a simple graph for which no perfect embedding exists, i.e., no embedding perfectly satisfies $u + \ell = v$ for all $(u, \ell, v) \in S$ and $u + \ell \neq v$ for $(u, \ell, v) \notin S$, for any choice of entity embeddings ($\mathbf{e}$ for $e \in E$) and relationship embeddings (ℓ for $\ell \in L$). Explain why this graph has no perfect embedding in this system, and what that means about the expressiveness of TransE embeddings. Assume the embeddings are in 2 dimensions ($k = 2$).

Hint: By expressiveness of TransE embeddings, we want you to talk about which type of relationships TransE can/cannot model.

2. Expressive Power of Knowledge Graph Embeddings (10 points)

TransE is a common method for learning representations of entities and relations in a knowledge graph. Given a triplet (h, ℓ, t) , where entities embedded as $\mathbf{h}$ and $\mathbf{t}$ are related by a relation embedded as ℓ , TransE trains entity and relation embeddings to make $\mathbf{h} + \ell$ close to $\mathbf{t}$.

There are some common patterns that relations form:

- Symmetry: A is married to B, and B is married to A.
- Inverse: A is the teacher of B, and B is the student of A. Note that teacher and student are two different relations and have their own embeddings.
- Composition: A is the son of B; C is the sister of B, then C is the aunt of A. Again, note that son, sister, and aunt are three different relations and have their own embeddings.

2.1 TransE Modeling (3 points)

For each of the above relational patterns, can TransE model them perfectly, such that $\mathbf{h} + \ell = \mathbf{t}$ for all relations? Explain why or why not.

Note: Zero embeddings for relations are undesirable, since they imply that two entities related by that relation are identical and not distinguishable.

2.2 RotatE Modeling (3 points)

Consider a new model, RotatE. Instead of training embeddings such that $\mathbf{h} + \ell \approx \mathbf{t}$, we train embeddings such that:

$$\mathbf{h} \circ \ell \approx \mathbf{t}.$$

Here, $\circ$ represents a rotation operation. You can think of $\mathbf{h}$ as a vector of dimension $2d$, representing d 2D points. ℓ is a d -dimensional vector specifying rotation angles. For all $i \in \{0, \dots, d-1\}$, each 2D point (h_{2i}, h_{2i+1}) is rotated clockwise by ℓ_i . Similar to TransE, the entity embeddings are also normalized to L2 norm 1. Can RotatE model the above three relational patterns perfectly? Why or why not?

2.3 Failure Cases (4 points)

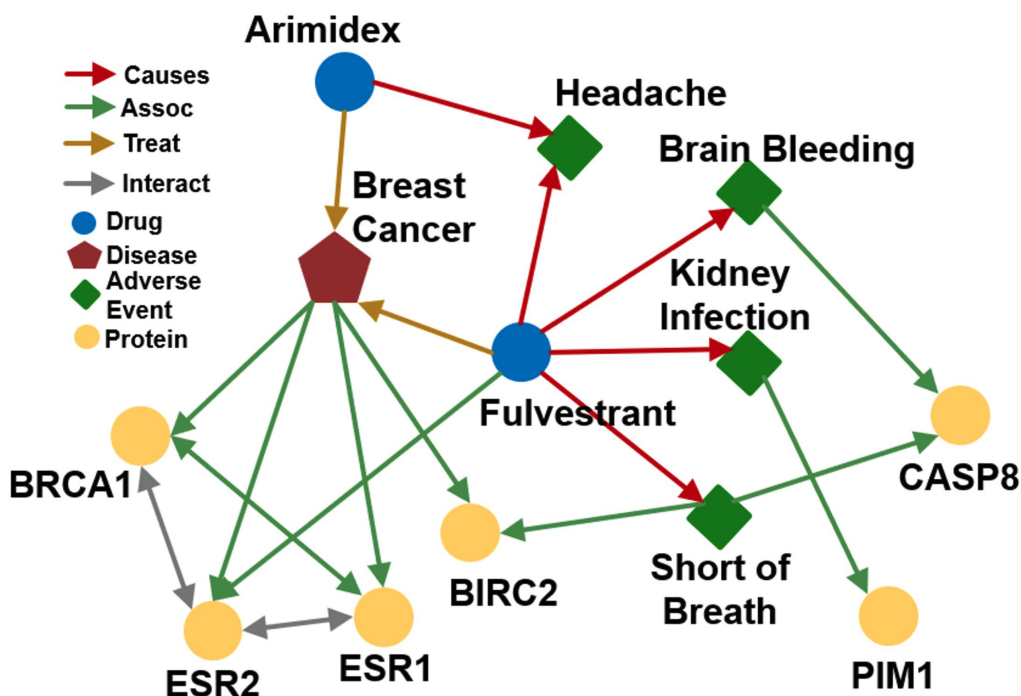
Give an example of a graph that RotatE cannot model. Can TransE model this graph? Assume that relation embeddings cannot be zero in either model.

3 Queries on Knowledge Graphs (15 points)

Knowledge graphs (KGs) can encode a wealth of information about the world. Beyond representing the information using knowledge graphs, we can often derive previously unknown insights about entities and relations in the graphs. In this question, we will explore different approaches for reasoning over knowledge graphs. Recall from lecture that we are interested in predicting tail nodes given (head, relation). We will use the same formulation throughout this question.

3.1 Path Queries on Complete KGs (3 points)

Consider the biomedicine knowledge graph from lecture. Assume the question of interest is: “What proteins are associated with diseases treated by Arimidex?” Write the question in query form (e.g., (e:AnchorEntity, (r:Relation))) and find the answer(s) to the query.



3.2 Conjunctive Queries on Complete KGs (2 points)

Consider the same biomedicine knowledge graph from before. Write a conjunctive query to which BIRC2 is the only answer using drugs as anchor entities. If such a query doesn't exist, provide a one-sentence explanation.

3.3 Incomplete KGs (2 points)

A major issue with direct traversals on knowledge graphs is that they are usually incomplete in reality. One solution is to encode entities, relations, and queries in an embedding space that meaningfully organizes information. We would then be able to impute missing relation links by considering all nearby points of the query embedding as answers to the query. We learned that TransE embeddings can be used for this. Can you come up with a way to adopt DistMult embeddings, which uses bilinear modeling, for answering path queries? If yes, describe in one or two sentences what can be modified from the TransE application. If no, provide a one-sentence explanation.

3.4 Query2box (8 points)

Query2box is an effective approach for answering complex conjunctive queries. Consider the following 2-dimensional embedding space. Assume that there are 7 entities $A, B, C, D, E, F, G \in V$, whose embeddings are shown below. There are 3 relations: R_1, R_2, R_3 . $R_1 \in R$ shifts the center of a box by $(0.25, 2)$ and increases the width and height of a box by $(0.5, 2)$. R_2 shifts the center of a box by $(1, 0)$ and increases the width and height of a box by $(1, 0)$. R_3 shifts the center of a box by $(-0.75, 1)$ and increases the width and height of a box by $(1.5, 3)$.

Use the Query2box projection operator to find the answers to the conjunctive query:

$$((e : A, (r : R_1, r : R_2)), (e : C, (r : R_3))).$$

Show your work.

Note: Shifting by a negative value means moving towards the left or bottom. Increasing the width and height by an amount means adding that amount in absolute value, not multiplying that amount as a factor. Assume that each path query starts with a box centered at the anchor entity with zero width and height.

